LifeLog: CI/CD with Jenkins, Docker, Docker Hub and Minikube

1. Project chosen

LifeLog is a small personal life tracker (Python 3.12, Flask, SQLite, gunicorn). Version 1 has three modules:

- **Habits**: add habits, daily check-in toggle, current and longest streak.
- **Expenses**: log amount, category and date; monthly total per category; monthly budget with percent used.
- **Dashboard**: today's habits, this month's spend vs budget, streak highlights.

The footer of every page shows the version, build number, git commit and pod name, so a pipeline run is visible in the browser. The UI uses an "origami fold" look (paper cards with folded corners, creases and layered shadows) in a single CSS file.

Run locally:

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
pytest -q
flask --app wsgi run        # http://127.0.0.1:5000
```

Layout: `app/` (factory, routes, `db.py`, pure logic in `habits_logic.py` and `expenses_logic.py`), `tests/`, `backend/Dockerfile`, `frontend/` (nginx image), `docker-compose.yml`, `Jenkinsfile`, `k8s/`.

Two images are built from one repo:

- **Backend** (`lifelog-backend`): Flask + gunicorn + SQLite, port 5000, 1 replica.
- **Frontend** (`lifelog-frontend`): nginx serving the static CSS and proxying all other requests to the backend, port 8080, 2 replicas. It is the only public entry point.

Run both locally: `docker compose up --build`, then open http://localhost:8080.

2. Pipeline stages

Stage	What it does
Checkout	Jenkins clones the GitHub repo and records the short commit id.
Build & Test	Creates a Python venv, installs `requirements.txt`, runs `pytest`. A failing test stops the pipeline.
Docker Build	Builds `saksham122004/lifelog-backend` and `saksham122004/lifelog-frontend` with build args `APP_VERSION`, `GIT_COMMIT`, `BUILD_NUMBER`; tags each `:<build number>` and `:latest`.
Push Image	Logs in to Docker Hub with credential `dockerhub-creds` (`--password-stdin`) and pushes both images, both tags.
Deploy to Minikube	Writes the new image tags into `k8s/backend.yaml` and `k8s/frontend.yaml`, runs `kubectl apply -f k8s/`, waits for both rollouts.

Stage	What it does
Verify	Lists pods/deployments/services, then curls `http://$(minikube ip):30080/health` and fails the build unless it returns 200.

`post { always }` runs `docker logout`; `post { failure }` prints `kubectl describe` and pod logs. A `pollSCM` trigger checks GitHub every 5 minutes as a fallback when no webhook exists.

3. How Jenkins was set up on AWS

1. EC2 Ubuntu 22.04, t3.medium, 30 GB disk. Security group: 22 (my IP), 8080 (Jenkins), 30080 (app).
2. Installed Docker, Java 17, Jenkins, kubectl, Minikube (docker driver) and git.
3. Added the `jenkins` user to the `docker` group and started Minikube as `jenkins`, so pipeline `kubectl` and `minikube` commands work.
4. Unlocked Jenkins at `http://<ec2-ip>:8080`, installed suggested plugins plus Docker Pipeline and Pipeline Stage View, created an admin user.
5. Added Docker Hub access token as credential `dockerhub-creds` (Username with password).
6. Created Pipeline job "lifelog-pipeline": Pipeline script from SCM, Git, this repo, branch `main`, path `Jenkinsfile`.

4. How the image was pushed

The pipeline builds both images on the EC2 host and pushes `saksham122004/lifelog-backend` and `saksham122004/lifelog-frontend`, each as `:<build>` and `:latest`, to Docker Hub. An access token is used instead of the account password: it can be revoked, scoped, and never exposes the real password.

5. How it was deployed and accessed

`k8s/backend.yaml`: 1 replica (SQLite lives in the pod, so it cannot be shared), probes on `/health`, the pod name injected through the downward API, and an internal ClusterIP service `backend:5000`. `k8s/frontend.yaml`: 2 replicas of nginx with probes on `/nginx-health`, and a NodePort service `lifelog` on 30080 (port 80 to container port 8080). The footer shows both the frontend pod (from the `X-Frontend-Pod` header nginx adds) and the backend pod.

Minikube with the docker driver runs inside a Docker network on the EC2 host, so `<ec2-public-ip>:30080` does not reach it directly. Pipeline checks use `minikube ip` from inside EC2. For the browser, forward the service on all interfaces:

```
sudo -u jenkins kubectl port-forward --address 0.0.0.0 svc/lifelog 30080:80
```

Then open `http://<ec2-public-ip>:30080`.

6. What changed for the second run

Branch `v2-goals` adds a **Goals** module (name, target, progress, deadline, percent complete) with its own tests. Merging it into `main` and pushing triggers a new build. The footer build number changes, the pod names change after the rolling update, and a Goals link appears in the navigation. The Jenkinsfile version also moves from 1.0.0 to 2.0.0.

Persistence note: SQLite lives inside the single backend pod, so data resets when that pod is replaced (and the backend cannot be scaled to 2 replicas without them each having their own data). A PersistentVolumeClaim (or a shared database) would fix this; it is left out to keep the assignment focused on the pipeline.

7. Troubleshooting

Symptom	Fix
`permission denied` on `/var/run/docker.sock`	`sudo usermod -aG docker jenkins` then `sudo systemctl restart jenkins`.
`minikube: host does not exist` / `kubectl` cannot connect in a build	Minikube is not running for the jenkins user: `sudo -u jenkins minikube start --driver=docker`.
`ImagePullBackOff`	Wrong Docker Hub username or the push failed; check the Push Image log and that both repos (`lifelog-backend`, `lifelog-frontend`) are public.
Frontend shows `502 Bad Gateway`	Backend pod not ready: `kubectl get pods`, `kubectl logs deploy/lifelog-backend-backend`.
`Verify` stage gets no 200	`kubectl get pods`, then `kubectl logs deploy/lifelog`; wait for readiness.
Port 30080 not reachable from the browser	Open the port in the security group and run the `port-forward` command above.
`python3 -m venv` fails	`sudo apt install -y python3-venv`.
Pipeline did not start on push	No webhook: wait for pollSCM (5 min) or click Build Now.

8. Screenshots

[Screenshot 1: Jenkins dashboard with the pipeline job]

[Screenshot 2: Pipeline stage view (all green)]

[Screenshot 3: Console Output of a successful run]

[Screenshot 4: Docker Hub repository with tags]

[Screenshot 5: kubectl get pods,deploy,svc]

[Screenshot 6: App in the browser (build 1)]

[Screenshot 7: App after the second run (Goals page, new build and pod)]